

COMPLETE PYTHON NOTES

Definitions, Syntax & Practical Core Examples

+ Practice Programs (Write a Program To...) with Input & Output

STUDENT HANDBOOK & WORKBOOK

TABLE OF CONTENTS

- 1. Basics: Introduction to Python**
- 2. Keywords**
- 3. Identifiers**
- 4. Comments**
- 5. Variables**
- 6. Operators (Arithmetic, Comparison, Logical, Assignment)**
- 7. Control Flow: Conditionals (if, elif, else)**
- 8. Loops and Control Keywords (while, for, break, continue)**
- 9. Data Structures (Lists, Tuples, Dictionaries, Sets, Strings)**
- 10. Functions (User-Defined, *args/**kwargs, Lambda, Comprehensions)**
- 11. Object-Oriented Programming (OOP Core Blueprint Traits)**
- 12. Advanced Topics (Files, Exceptions, Generators, Decorators, re)**
- 13. Interview Programs (40 Mixed Practice Questions)**

Note: Every topic and sub-topic below is followed by 5 practice programs in the format "Write a program to..." with Input and Output shown. A bonus Interview Programs section with 40 total questions appears at the end.

1. Basics: Introduction to Python

Python is a high-level, interpreted programming language known for its clear syntax and great readability. It is widely used in web development, data science, artificial intelligence, and automation tasks.

Syntax:

```
print("Your Message Here")
```

Example Program:

```
# This program prints a welcoming message to the screen
print("Hello, Welcome to the world of Python programming!")
```

Output:

```
Hello, Welcome to the world of Python programming!
```

PRACTICE PROGRAMS - Basics: Introduction to Python

Program 1: Write a program to print your name and age.

Input:

None (values are defined inside the program)

Program / Code:

```
name = "Praveen"
age = 21
print("Name:", name)
print("Age:", age)
```

Output:

```
Name: Praveen
Age: 21
```

Program 2: Write a program to print a multi-line message using multiple print statements.

Input:

None

Program / Code:

```
print("Welcome to Python")
print("This is line two")
print("This is line three")
```

Output:

```
Welcome to Python
This is line two
This is line three
```

Program 3: Write a program to display the sum of two hard-coded numbers.

Input:

None

Program / Code:

```
num1 = 12
num2 = 18
print("Sum is:", num1 + num2)
```

Output:

Sum is: 30

Program 4: Write a program to print a message using single and double quotes.

Input:

None

Program / Code:

```
print('Python uses single quotes too')
print("Double quotes also work fine")
```

Output:

Python uses single quotes too
Double quotes also work fine

Program 5: Write a program to print the Python version installed on the system.

Input:

None

Program / Code:

```
import sys
print("Python Version:", sys.version)
```

Output:

Python Version: 3.12.x (...) [output depends on installed version]

2. Keywords

Keywords are reserved words in Python that have fixed meanings and cannot be used as regular identifiers. They form the core vocabulary of the language to build control logic and definitions.

Syntax:

```
True, False, None, and, or, not, if, else, elif, for, while,
break, continue, def, return, class, import
```

Example Program:

```
# Using True, if, and else keywords to control program execution flow
condition_flag = True
if condition_flag:
    print("The keyword condition is satisfied.")
```

Output:

```
The keyword condition is satisfied.
```

PRACTICE PROGRAMS - Keywords

Program 1: Write a program to check if a word is a Python keyword.

Input:

```
word = "for"
```

Program / Code:

```
import keyword
word = "for"
print(keyword.iskeyword(word))
```

Output:

```
True
```

Program 2: Write a program to print all Python keywords using the keyword module.

Input:

```
None
```

Program / Code:

```
import keyword
print(keyword.kwlist)
```

Output:

```
['False', 'None', 'True', 'and', 'as', 'assert', ... , 'while',
'with', 'yield']
```

Program 3: Write a program to count the number of keywords in Python.

Input:

None

Program / Code:

```
import keyword
print("Total keywords:", len(keyword.kwlist))
```

Output:

Total keywords: 36

Program 4: Write a program to demonstrate use of True and False keywords in a condition.

Input:

is_logged_in = False

Program / Code:

```
is_logged_in = False
if is_logged_in:
    print("Access granted")
else:
    print("Access denied")
```

Output:

Access denied

Program 5: Write a program to demonstrate the use of the None keyword.

Input:

None

Program / Code:

```
result = None
if result is None:
    print("No value assigned yet")
```

Output:

No value assigned yet

3. Identifiers

Identifiers are user-defined names given to entities like variables, functions, classes, and modules to recognize them distinctly. They must start with a letter or an underscore, followed by letters, digits, or underscores.

Syntax:

```
valid_name = value
_valid_name2 = value
```

Example Program:

```
# Assigning values to valid identifiers using alphanumeric
characters and underscores
student_age = 21
_user_score = 94.5
print(student_age, _user_score)
```

Output:

```
21 94.5
```

PRACTICE PROGRAMS - Identifiers

Program 1: Write a program to create variables with valid identifier names and print their values.

Input:

```
None
```

Program / Code:

```
city_name = "Bengaluru"
pin_code = 560001
print(city_name, pin_code)
```

Output:

```
Bengaluru 560001
```

Program 2: Write a program to check if a given string is a valid identifier.

Input:

```
text = "student_age"
```

Program / Code:

```
text = "student_age"
print(text.isidentifier())
```

Output:

```
True
```

Program 3: Write a program to demonstrate an identifier starting with an underscore.

Input:

None

Program / Code:

```
_temp_value = 100  
print(_temp_value)
```

Output:

100

Program 4: Write a program to demonstrate case sensitivity of identifiers.

Input:

None

Program / Code:

```
marks = 80  
Marks = 90  
print(marks, Marks)
```

Output:

80 90

Program 5: Write a program to create multiple variables in a single line using identifiers.

Input:

None

Program / Code:

```
a, b, c = 10, 20, 30  
print(a, b, c)
```

Output:

10 20 30

4. Comments

Comments are non-executable explanatory notes written within the code to help developers read and understand the logic. Python uses the hash symbol for single lines or triple quotes for multi-line annotations.

Syntax:

```
# Single line comment  
''' Multi-line comment block '''
```

Example Program:

```
# This entire statement is a single-line comment ignored by the
Python compiler
print("Comments are hidden instructions")
```

Output:

```
Comments are hidden instructions
```

PRACTICE PROGRAMS - Comments

Program 1: Write a program using a single-line comment to explain the code.

Input:

```
None
```

Program / Code:

```
# Print the greeting message
print("Hello there!")
```

Output:

```
Hello there!
```

Program 2: Write a program using a multi-line comment block.

Input:

```
None
```

Program / Code:

```
'''
This program adds two numbers
and prints the result
'''
print(5 + 7)
```

Output:

```
12
```

Program 3: Write a program to add inline comments after code statements.

Input:

None

Program / Code:

```
price = 500 # price of the item
tax = 50 # tax amount
print(price + tax) # total payable
```

Output:

550

Program 4: Write a program to comment out a line of code temporarily.

Input:

None

Program / Code:

```
print("This line runs")
# print("This line is disabled")
```

Output:

This line runs

Program 5: Write a program with comments explaining each step of adding two numbers.

Input:

None

Program / Code:

```
# Step 1: take two numbers
a = 4
b = 6
# Step 2: add them
total = a + b
# Step 3: display result
print(total)
```

Output:

10

5. Variables

Variables are named storage locations or containers used to hold data values that can change during execution. Python allocates memory dynamically, so you do not need to declare data types explicitly.

Syntax:

```
variable_name = data_value
```

Example Program:

```
# Storing different types of data values inside named variable containers
user_name = "Praveen"
user_rank = 10
print(user_name, user_rank)
```

Output:

```
Praveen 10
```

PRACTICE PROGRAMS - Variables

Program 1: Write a program to declare and print variables of different types.

Input:

```
None
```

Program / Code:

```
name = "Asha"
age = 25
height = 5.4
print(name, age, height)
```

Output:

```
Asha 25 5.4
```

Program 2: Write a program to swap the values of two variables.

Input:

```
a = 5, b = 9
```

Program / Code:

```
a = 5
b = 9
a, b = b, a
print("a =", a, "b =", b)
```

Output:

```
a = 9 b = 5
```

Program 3: Write a program to assign multiple values to multiple variables in one line.

Input:

None

Program / Code:

```
x, y, z = 1, 2, 3  
print(x, y, z)
```

Output:

1 2 3

Program 4: Write a program to assign the same value to multiple variables.

Input:

None

Program / Code:

```
p = q = r = 50  
print(p, q, r)
```

Output:

50 50 50

Program 5: Write a program to check the type of a variable using type().

Input:

score = 88.5

Program / Code:

```
score = 88.5  
print(type(score))
```

Output:

<class 'float'>

6.1. Operators - Arithmetic Operators

Arithmetic operators are standard mathematical symbols used to carry out numeric computations like addition, subtraction, multiplication, and division.

Syntax:

```
+, -, *, /, %, **, //
```

Example Program:

```
# Performing simple math calculations using numeric operators
sum_result = 15 + 5
power_result = 2 ** 3
print(sum_result, power_result)
```

Output:

```
20 8
```

PRACTICE PROGRAMS - Operators - Arithmetic Operators

Program 1: Write a program to add two numbers.

Input:

```
a = 25, b = 15
```

Program / Code:

```
a = 25
b = 15
print("Sum:", a + b)
```

Output:

```
Sum: 40
```

Program 2: Write a program to find the remainder of two numbers.

Input:

```
17, 4
```

Program / Code:

```
print("Remainder:", 17 % 4)
```

Output:

```
Remainder: 1
```

Program 3: Write a program to calculate the power of a number.

Input:

```
3, 4
```

Program / Code:

```
print("Power:", 3 ** 4)
```

Output:

```
Power: 81
```

Program 4: Write a program to perform floor division of two numbers.

Input:

```
29, 4
```

Program / Code:

```
print("Floor Division:", 29 // 4)
```

Output:

```
Floor Division: 7
```

Program 5: Write a program to calculate the area of a rectangle using multiplication.

Input:

```
length = 12, breadth = 5
```

Program / Code:

```
length = 12  
breadth = 5  
print("Area:", length * breadth)
```

Output:

```
Area: 60
```

6.2. Operators - Comparison Operators

Comparison operators evaluate relationships between two operands and return a Boolean response of either True or False based on equality or size.

Syntax:

```
==, !=, >, <, >=, <=
```

Example Program:

```
# Checking balance boundaries using conditional comparison tokens
check_equality = (10 == 10)
check_greater = (5 > 12)
print(check_equality, check_greater)
```

Output:

```
True False
```

PRACTICE PROGRAMS - Operators - Comparison Operators

Program 1: Write a program to compare two numbers using comparison operators.

Input:

```
a = 10, b = 20
```

Program / Code:

```
a = 10
b = 20
print(a < b)
```

Output:

```
True
```

Program 2: Write a program to check if two strings are equal.

Input:

```
s1 = "python", s2 = "Python"
```

Program / Code:

```
s1 = "python"
s2 = "Python"
print(s1 == s2)
```

Output:

```
False
```

Program 3: Write a program to check if a number is greater than another.

Input:

```
45, 30
```

Program / Code:

```
print(45 > 30)
```

Output:

```
True
```

Program 4: Write a program to check if a number is not equal to another.

Input:

8, 8

Program / Code:

```
print(8 != 8)
```

Output:

False

Program 5: Write a program to compare the ages of two people.

Input:

age1 = 24, age2 = 30

Program / Code:

```
age1 = 24
age2 = 30
print("age1 is younger:", age1 < age2)
```

Output:

age1 is younger: True

6.3. Operators - Logical Operators

Logical operators combine conditional assertions to evaluate compound Boolean conditions and direct logical outcomes.

Syntax:

and, or, not

Example Program:

```
# Validating relational equations using Boolean truth table
operators
combined_logic = (5 > 2) and (10 < 20)
inverted_logic = not(5 == 5)
print(combined_logic, inverted_logic)
```

Output:

True False

PRACTICE PROGRAMS - Operators - Logical Operators

Program 1: Write a program to check eligibility using the 'and' operator.

Input:

```
age = 20, has_id = True
```

Program / Code:

```
age = 20
has_id = True
print(age >= 18 and has_id)
```

Output:

```
True
```

Program 2: Write a program to check eligibility using the 'or' operator.

Input:

```
has_ticket = False, is_vip = True
```

Program / Code:

```
has_ticket = False
is_vip = True
print(has_ticket or is_vip)
```

Output:

```
True
```

Program 3: Write a program to invert a boolean value using the 'not' operator.

Input:

```
is_raining = False
```

Program / Code:

```
is_raining = False
print(not is_raining)
```

Output:

```
True
```

Program 4: Write a program to check if a number is within a range using logical operators.

Input:

```
num = 15
```

Program / Code:

```
num = 15
print(10 < num < 20)
```

Output:

```
True
```

Program 5: Write a program to validate login credentials using logical operators.

Input:

```
username = "admin", password = "1234"
```

Program / Code:

```
username = "admin"
password = "1234"
print(username == "admin" and password == "1234")
```

Output:

```
True
```

6.4. Operators - Assignment Operators

Assignment operators give values to variables or update their existing storage values in combination with mathematical operations.

Syntax:

```
=, +=, -=, *=, /=
```

Example Program:

```
# Modifying values of variables using combined operations and
shortcuts
score_tracker = 50
score_tracker += 10
print(score_tracker)
```

Output:

```
60
```

PRACTICE PROGRAMS - Operators - Assignment Operators

Program 1: Write a program to increment a variable using +=.

Input:

```
total = 100
```

Program / Code:

```
total = 100
total += 25
print(total)
```

Output:

```
125
```

Program 2: Write a program to decrement a variable using -=.

Input:

```
stock = 80
```

Program / Code:

```
stock = 80
stock -= 15
print(stock)
```

Output:

```
65
```

Program 3: Write a program to multiply a variable using *=.

Input:

```
value = 6
```

Program / Code:

```
value = 6
value *= 4
print(value)
```

Output:

```
24
```

Program 4: Write a program to divide a variable using /=.

Input:

```
amount = 90
```

Program / Code:

```
amount = 90  
amount /= 3  
print(amount)
```

Output:

```
30.0
```

Program 5: Write a program to demonstrate the use of //=.

Input:

```
num = 29
```

Program / Code:

```
num = 29  
num //= 5  
print(num)
```

Output:

```
5
```

7.1. Control Flow: Conditionals - 'if' Statement

The 'if' statement runs a specific block of target code only when the provided evaluation expression tests as True.

Syntax:

```
if condition:
    # Code block to execute
```

Example Program:

```
# Checking single affirmative parameters using the simple if
structure
current_marks = 85
if current_marks >= 75:
    print("You passed with distinction marks!")
```

Output:

```
You passed with distinction marks!
```

PRACTICE PROGRAMS - Control Flow: Conditionals - 'if' Statement

Program 1: Write a program to check if a number is positive.

Input:

```
num = 7
```

Program / Code:

```
num = 7
if num > 0:
    print("Positive number")
```

Output:

```
Positive number
```

Program 2: Write a program to check if a number is even.

Input:

```
num = 14
```

Program / Code:

```
num = 14
if num % 2 == 0:
    print("Even number")
```

Output:

```
Even number
```

Program 3: Write a program to check if a person is eligible to vote.

Input:

```
age = 19
```

Program / Code:

```
age = 19
if age >= 18:
    print("Eligible to vote")
```

Output:

```
Eligible to vote
```

Program 4: Write a program to check if a string is empty.

Input:

```
text = ""
```

Program / Code:

```
text = ""
if not text:
    print("String is empty")
```

Output:

```
String is empty
```

Program 5: Write a program to check if a number is divisible by 5.

Input:

```
num = 45
```

Program / Code:

```
num = 45
if num % 5 == 0:
    print("Divisible by 5")
```

Output:

```
Divisible by 5
```

7.2. Control Flow: Conditionals - 'elif' Statement

The 'elif' (else-if) statement tests alternative condition scenarios sequentially when the original 'if' logic fails.

Syntax:

```
elif alternate_condition:
    # Code block to execute
```

Example Program:

```
# Layering secondary logical tests when prior conditions fail
current_marks = 65
if current_marks >= 75:
    print("Distinction grade")
elif current_marks >= 60:
    print("First class grade")
```

Output:

First class grade

PRACTICE PROGRAMS - Control Flow: Conditionals - 'elif' Statement

Program 1: Write a program to grade a student based on marks.

Input:

marks = 72

Program / Code:

```
marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

Output:

Grade B

Program 2: Write a program to categorize a number as small, medium or large.

Input:

num = 55

Program / Code:

```
num = 55
if num < 10:
    print("Small")
elif num < 100:
    print("Medium")
else:
    print("Large")
```

Output:

Medium

Program 3: Write a program to print the day name for a given day number.

Input:

```
day = 3
```

Program / Code:

```
day = 3
if day == 1:
    print("Monday")
elif day == 2:
    print("Tuesday")
elif day == 3:
    print("Wednesday")
else:
    print("Other day")
```

Output:

```
Wednesday
```

Program 4: Write a program to determine ticket price based on age group.

Input:

```
age = 10
```

Program / Code:

```
age = 10
if age < 5:
    print("Free entry")
elif age <= 12:
    print("Half ticket")
else:
    print("Full ticket")
```

Output:

```
Half ticket
```

Program 5: Write a program to classify BMI category.

Input:

```
bmi = 27
```

Program / Code:

```
bmi = 27
if bmi < 18.5:
    print("Underweight")
elif bmi < 25:
    print("Normal")
elif bmi < 30:
    print("Overweight")
else:
    print("Obese")
```

Output:

```
Overweight
```

7.3. Control Flow: Conditionals - 'else' Statement

The 'else' block serves as a default catch-all that runs when none of the preceding conditional statements are True.

Syntax:

```
else:
    # Code block to execute
```

Example Program:

```
# Handling fallback outcomes when all previous condition
validations fail
current_marks = 45
if current_marks >= 50:
    print("Passed successfully")
else:
    print("Failed the examination")
```

Output:

```
Failed the examination
```

PRACTICE PROGRAMS - Control Flow: Conditionals - 'else' Statement

Program 1: Write a program to check if a number is even or odd.

Input:

```
num = 9
```

Program / Code:

```
num = 9
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Output:

```
Odd
```

Program 2: Write a program to check if a person is eligible to vote or not.

Input:

```
age = 15
```

Program / Code:

```
age = 15
if age >= 18:
    print("Eligible")
else:
    print("Not eligible")
```

Output:

```
Not eligible
```

Program 3: Write a program to check if a year is a leap year or not.

Input:

```
year = 2024
```

Program / Code:

```
year = 2024
if year % 400 == 0 or (year % 4 == 0 and year % 100 != 0):
    print("Leap year")
else:
    print("Not a leap year")
```

Output:

```
Leap year
```

Program 4: Write a program to check pass or fail based on marks.

Input:

```
marks = 38
```

Program / Code:

```
marks = 38
if marks >= 40:
    print("Pass")
else:
    print("Fail")
```

Output:

```
Fail
```

Program 5: Write a program to check if a number is positive or negative.

Input:

```
num = -6
```

Program / Code:

```
num = -6
if num >= 0:
    print("Positive")
else:
    print("Negative")
```

Output:

```
Negative
```

8.1. Loops and Control Keywords - 'while' Loop

A 'while' loop runs a target block of instructions repeatedly as long as its guiding conditional check remains True.

Syntax:

```
while condition:
    # Code block to repeat
```

Example Program:

```
# Counting up step-by-step using continuous loop test parameters
step_counter = 1
while step_counter <= 3:
    print("Loop iteration step:", step_counter)
    step_counter += 1
```

Output:

```
Loop iteration step: 1
Loop iteration step: 2
Loop iteration step: 3
```

PRACTICE PROGRAMS - Loops and Control Keywords - 'while' Loop

Program 1: Write a program to print numbers from 1 to 5 using a while loop.

Input:

None

Program / Code:

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

Output:

```
1
2
3
4
5
```

Program 2: Write a program to calculate the factorial of a number using a while loop.

Input:

```
n = 5
```

Program / Code:

```
n = 5
fact = 1
while n > 0:
    fact *= n
    n -= 1
print(fact)
```

Output:

```
120
```

Program 3: Write a program to print the sum of the first n natural numbers using a while loop.

Input:

```
n = 5
```

Program / Code:

```
n = 5
total = 0
i = 1
while i <= n:
    total += i
    i += 1
print(total)
```

Output:

```
15
```

Program 4: Write a program to reverse a number using a while loop.

Input:

```
num = 1234
```

Program / Code:

```
num = restart = 1234
rev = 0
while num > 0:
    rev = rev * 10 + num % 10
    num //= 10
print(rev)
```

Output:

```
4321
```

Program 5: Write a program to print a countdown from 5 to 1.

Input:

```
None
```

Program / Code:

```
n = 5
while n >= 1:
    print(n)
    n -= 1
```

Output:

```
5
4
3
2
1
```

8.2. Loops and Control Keywords - 'for' Loop

The 'for' loop iterates over sequential structures like lists, tuples, or ranges, running the block for each element.

Syntax:

```
for item in sequence:
    # Code block to repeat
```

Example Program:

```
# Reading collections using direct element iteration controls
for current_number in range(1, 4):
    print("Sequence value index is:", current_number)
```

Output:

```
Sequence value index is: 1
Sequence value index is: 2
Sequence value index is: 3
```

PRACTICE PROGRAMS - Loops and Control Keywords - 'for' Loop

Program 1: Write a program to print numbers from 1 to 5 using a for loop.

Input:

None

Program / Code:

```
for i in range(1, 6):
    print(i)
```

Output:

```
1
2
3
4
5
```

Program 2: Write a program to print elements of a list using a for loop.

Input:

```
fruits = ["apple", "mango", "grapes"]
```

Program / Code:

```
fruits = ["apple", "mango", "grapes"]
for f in fruits:
    print(f)
```

Output:

```
apple
mango
grapes
```

Program 3: Write a program to calculate the sum of numbers in a list.

Input:

```
nums = [4, 8, 6, 2]
```

Program / Code:

```
nums = [4, 8, 6, 2]
total = 0
for n in nums:
    total += n
print(total)
```

Output:

```
20
```

Program 4: Write a program to print the multiplication table of a number.

Input:

```
num = 5
```

Program / Code:

```
num = 5
for i in range(1, 6):
    print(num, "x", i, "=", num * i)
```

Output:

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
```

Program 5: Write a program to count vowels in a string using a for loop.

Input:

```
text = "python programming"
```

Program / Code:

```
text = "python programming"
count = 0
for ch in text:
    if ch in "aeiou":
        count += 1
print(count)
```

Output:

```
4
```

8.3. Loops and Control Keywords - 'break' Keyword

The 'break' statement terminates the active execution loop early and shifts the program flow to the next line outside the loop.

Syntax:

```
break
```

Example Program:

```
# Exiting an active loop block instantly using the explicit break statement
for item_val in range(1, 10):
    if item_val == 4:
        break
    print("Item index counter:", item_val)
```

Output:

```
Item index counter: 1
Item index counter: 2
Item index counter: 3
```

PRACTICE PROGRAMS - Loops and Control Keywords - 'break' Keyword

Program 1: Write a program to stop a loop when a specific number is found.

Input:

```
None
```

Program / Code:

```
for i in range(1, 10):
    if i == 6:
        print("Found 6")
        break
```

Output:

```
Found 6
```

Program 2: Write a program to break a loop when a negative number is encountered.

Input:

```
nums = [3, 7, -2, 9]
```

Program / Code:

```
nums = [3, 7, -2, 9]
for n in nums:
    if n < 0:
        break
    print(n)
```

Output:

```
3
7
```

Program 3: Write a program to search for an element in a list using break.

Input:

```
items = ["pen", "book", "bag"]
```

Program / Code:

```
items = ["pen", "book", "bag"]
for i in items:
    if i == "book":
        print("Item found")
        break
```

Output:

```
Item found
```

Program 4: Write a program to break out of a loop at the fifth iteration.

Input:

```
None
```

Program / Code:

```
for i in range(1, 20):
    if i == 5:
        break
    print(i)
```

Output:

```
1
2
3
4
```

Program 5: Write a program to stop printing numbers when 7 is reached.

Input:

None

Program / Code:

```
for i in range(1, 10):
    if i == 7:
        break
    print(i)
```

Output:

```
1
2
3
4
5
6
```

8.4. Loops and Control Keywords - 'continue' Keyword

The 'continue' statement skips the remaining code in the current iteration and jumps directly to the loop's next evaluation.

Syntax:

continue

Example Program:

```
# Skipping specific target items while keeping the loop running
normally
for numeric_val in range(1, 5):
    if numeric_val == 3:
        continue
    print("Active structural digit:", numeric_val)
```

Output:

```
Active structural digit: 1
Active structural digit: 2
Active structural digit: 4
```

PRACTICE PROGRAMS - Loops and Control Keywords - 'continue' Keyword

Program 1: Write a program to skip printing even numbers using continue.

Input:

None

Program / Code:

```
for i in range(1, 8):  
    if i % 2 == 0:  
        continue  
    print(i)
```

Output:

```
1  
3  
5  
7
```

Program 2: Write a program to skip a specific element in a list.

Input:

```
items = [1, 2, 3, 4, 5]
```

Program / Code:

```
items = [1, 2, 3, 4, 5]  
for i in items:  
    if i == 3:  
        continue  
    print(i)
```

Output:

```
1  
2  
4  
5
```

Program 3: Write a program to print only odd numbers using continue.

Input:

None

Program / Code:

```
for i in range(1, 10):  
    if i % 2 == 0:  
        continue  
    print(i)
```

Output:

```
1  
3  
5  
7  
9
```

Program 4: Write a program to skip negative numbers while summing a list.

Input:

nums = [4, -3, 6, -1, 2]

Program / Code:

```
nums = [4, -3, 6, -1, 2]  
total = 0  
for n in nums:  
    if n < 0:  
        continue  
    total += n  
print(total)
```

Output:

12

Program 5: Write a program to skip vowels while printing characters of a string.

Input:

```
text = "hello"
```

Program / Code:

```
text = "hello"
for ch in text:
    if ch in "aeiou":
        continue
    print(ch)
```

Output:

```
h
l
l
```

9.1. Data Structures - Lists

Lists are mutable, ordered collections of items enclosed in square brackets that allow duplicate records and direct index changes.

Syntax:

```
list_name = [item1, item2, item3]
```

Example Program:

```
# Initializing, printing, and modifying values inside a standard list
fruits_list = ["apple", "banana"]
fruits_list.append("cherry")
print("Updated target list object:", fruits_list)
```

Output:

```
Updated target list object: ['apple', 'banana', 'cherry']
```

PRACTICE PROGRAMS - Data Structures - Lists

Program 1: Write a program to create a list and print its elements.

Input:

```
None
```

Program / Code:

```
colors = ["red", "green", "blue"]
print(colors)
```

Output:

```
['red', 'green', 'blue']
```

Program 2: Write a program to add an element to a list.

Input:

```
nums = [1, 2, 3]
```

Program / Code:

```
nums = [1, 2, 3]
nums.append(4)
print(nums)
```

Output:

```
[1, 2, 3, 4]
```

Program 3: Write a program to remove an element from a list.

Input:

```
items = [10, 20, 30]
```

Program / Code:

```
items = [10, 20, 30]
items.remove(20)
print(items)
```

Output:

```
[10, 30]
```

Program 4: Write a program to find the length of a list.

Input:

```
data = [5, 8, 2, 9]
```

Program / Code:

```
data = [5, 8, 2, 9]
print(len(data))
```

Output:

```
4
```

Program 5: Write a program to sort a list of numbers.

Input:

```
nums = [8, 3, 6, 1]
```

Program / Code:

```
nums = [8, 3, 6, 1]
nums.sort()
print(nums)
```

Output:

```
[1, 3, 6, 8]
```

9.2. Data Structures - Tuples

Tuples are immutable, ordered sequences wrapped inside standard parentheses that protect data from accidental updates after creation.

Syntax:

```
tuple_name = (item1, item2, item3)
```

Example Program:

```
# Setting up historical reference storage points using strict
unchangeable tuples
coordinate_point = (12.5, 45.8)
print("Fixed system coordinate details:", coordinate_point[0])
```

Output:

```
Fixed system coordinate details: 12.5
```

PRACTICE PROGRAMS - Data Structures - Tuples

Program 1: Write a program to create a tuple and access its elements.

Input:

```
point = (3, 4, 5)
```

Program / Code:

```
point = (3, 4, 5)
print(point[0], point[2])
```

Output:

```
3 5
```

Program 2: Write a program to find the length of a tuple.

Input:

```
data = (1, 2, 3, 4)
```

Program / Code:

```
data = (1, 2, 3, 4)
print(len(data))
```

Output:

```
4
```

Program 3: Write a program to count occurrences of an element in a tuple.

Input:

```
nums = (1, 2, 2, 3, 2)
```

Program / Code:

```
nums = (1, 2, 2, 3, 2)
print(nums.count(2))
```

Output:

```
3
```

Program 4: Write a program to concatenate two tuples.

Input:

```
t1 = (1, 2), t2 = (3, 4)
```

Program / Code:

```
t1 = (1, 2)
t2 = (3, 4)
print(t1 + t2)
```

Output:

```
(1, 2, 3, 4)
```

Program 5: Write a program to convert a list into a tuple.

Input:

```
lst = [1, 2, 3]
```

Program / Code:

```
lst = [1, 2, 3]
tup = tuple(lst)
print(tup)
```

Output:

```
(1, 2, 3)
```

9.3. Data Structures - Dictionaries

Dictionaries are unordered, mutable structures that store elements as key-value pairs inside curly braces for fast lookups.

Syntax:

```
dict_name = {"key": "value"}
```

Example Program:

```
# Creating profile catalogs using readable, paired relational
data links
user_profile = {"id": 101, "role": "Admin"}
print("Profile assigned role designation:", user_profile["role"])
```

Output:

```
Profile assigned role designation: Admin
```

PRACTICE PROGRAMS - Data Structures - Dictionaries

Program 1: Write a program to create a dictionary and access its values.

Input:

```
student = {"name": "Riya", "age": 20}
```

Program / Code:

```
student = {"name": "Riya", "age": 20}  
print(student["name"])
```

Output:

```
Riya
```

Program 2: Write a program to add a new key-value pair to a dictionary.

Input:

```
data = {"a": 1}
```

Program / Code:

```
data = {"a": 1}  
data["b"] = 2  
print(data)
```

Output:

```
{'a': 1, 'b': 2}
```

Program 3: Write a program to remove a key from a dictionary.

Input:

```
data = {"x": 1, "y": 2}
```

Program / Code:

```
data = {"x": 1, "y": 2}  
del data["x"]  
print(data)
```

Output:

```
{'y': 2}
```

Program 4: Write a program to loop through keys and values of a dictionary.

Input:

```
marks = {"Math": 90, "Science": 85}
```

Program / Code:

```
marks = {"Math": 90, "Science": 85}
for k, v in marks.items():
    print(k, ":", v)
```

Output:

```
Math : 90
Science : 85
```

Program 5: Write a program to check if a key exists in a dictionary.

Input:

```
data = {"id": 1}
```

Program / Code:

```
data = {"id": 1}
print("id" in data)
```

Output:

```
True
```

9.4. Data Structures - Sets

Sets are unordered collections of unique elements wrapped in curly braces that filter out duplicates and handle mathematical operations.

Syntax:

```
set_name = {item1, item2}
```

Example Program:

```
# Removing duplicates automatically using unique item collections
unique_digits = {1, 2, 2, 3}
print("Clean unique set collection values:", unique_digits)
```

Output:

```
Clean unique set collection values: {1, 2, 3}
```

PRACTICE PROGRAMS - Data Structures - Sets

Program 1: Write a program to create a set and remove duplicates automatically.

Input:

```
nums = {1, 2, 2, 3, 3, 3}
```

Program / Code:

```
nums = {1, 2, 2, 3, 3, 3}  
print(nums)
```

Output:

```
{1, 2, 3}
```

Program 2: Write a program to add an element to a set.

Input:

```
s = {1, 2, 3}
```

Program / Code:

```
s = {1, 2, 3}  
s.add(4)  
print(s)
```

Output:

```
{1, 2, 3, 4}
```

Program 3: Write a program to find the union of two sets.

Input:

```
a = {1, 2, 3}, b = {3, 4, 5}
```

Program / Code:

```
a = {1, 2, 3}  
b = {3, 4, 5}  
print(a | b)
```

Output:

```
{1, 2, 3, 4, 5}
```

Program 4: Write a program to find the intersection of two sets.

Input:

```
a = {1, 2, 3}, b = {2, 3, 4}
```

Program / Code:

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a & b)
```

Output:

```
{2, 3}
```

Program 5: Write a program to remove an element from a set.

Input:

```
s = {10, 20, 30}
```

Program / Code:

```
s = {10, 20, 30}
s.remove(20)
print(s)
```

Output:

```
{10, 30}
```

9.5. Data Structures - Strings

Strings are immutable sequences of text characters wrapped inside quotation marks, equipped with built-in manipulation methods.

Syntax:

```
string_name = "Text Characters Here"
```

Example Program:

```
# Executing character string edits using built-in manipulation
methods
greeting_text = "hello"
print("Capitalized string modification:", greeting_text.upper())
```

Output:

```
Capitalized string modification: HELLO
```

PRACTICE PROGRAMS - Data Structures - Strings

Program 1: Write a program to reverse a string.

Input:

```
text = "python"
```

Program / Code:

```
text = "python"  
print(text[::-1])
```

Output:

```
nohtyp
```

Program 2: Write a program to count the number of characters in a string.

Input:

```
text = "hello world"
```

Program / Code:

```
text = "hello world"  
print(len(text))
```

Output:

```
11
```

Program 3: Write a program to convert a string to lowercase.

Input:

```
text = "PYTHON"
```

Program / Code:

```
text = "PYTHON"  
print(text.lower())
```

Output:

```
python
```

Program 4: Write a program to check if a string starts with a given word.

Input:

```
text = "Hello World"
```

Program / Code:

```
text = "Hello World"  
print(text.startswith("Hello"))
```

Output:

```
True
```

Program 5: Write a program to replace a word in a string.

Input:

```
text = "I like Java"
```

Program / Code:

```
text = "I like Java"  
print(text.replace("Java", "Python"))
```

Output:

```
I like Python
```

10.1. Functions - User-Defined Functions

User-defined functions are modular, reusable blocks of code declared with the 'def' keyword to perform a specific task when called.

Syntax:

```
def function_name(parameters):  
    return result
```

Example Program:

```
# Declaring structural functional steps that can be run  
repeatedly  
def calculate_square(input_val):  
    return input_val * input_val  
print("Function call computation output:", calculate_square(5))
```

Output:

```
Function call computation output: 25
```

PRACTICE PROGRAMS - Functions - User-Defined Functions

Program 1: Write a program to define a function to add two numbers.

Input:

```
a = 6, b = 9
```

Program / Code:

```
def add(a, b):  
    return a + b  
print(add(6, 9))
```

Output:

```
15
```

Program 2: Write a program to define a function to check if a number is prime.

Input:

```
n = 7
```

Program / Code:

```
def is_prime(n):  
    if n < 2:  
        return False  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True  
print(is_prime(7))
```

Output:

```
True
```

Program 3: Write a program to define a function that returns the maximum of two numbers.

Input:

```
a = 12, b = 20
```

Program / Code:

```
def maximum(a, b):  
    return a if a > b else b  
print(maximum(12, 20))
```

Output:

```
20
```

Program 4: Write a program to define a function with a default parameter value.

Input:

```
None
```

Program / Code:

```
def greet(name="Guest"):  
    return "Hello " + name  
print(greet())  
print(greet("Amit"))
```

Output:

```
Hello Guest  
Hello Amit
```

Program 5: Write a program to define a function to calculate simple interest.

Input:

```
p=1000, r=5, t=2
```

Program / Code:

```
def simple_interest(p, r, t):  
    return (p * r * t) / 100  
print(simple_interest(1000, 5, 2))
```

Output:

```
100.0
```

10.2. Functions - *args and **kwargs

*args accepts a variable number of positional arguments as a tuple, while **kwargs accepts a variable number of keyword arguments as a dictionary.

Syntax:

```
def function_name(*args, **kwargs):
```

Example Program:

```
# Capturing indefinite inputs using flexible parameters  
def display_flexible_inputs(*args, **kwargs):  
    print("Positional arguments tuple:", args)  
    print("Keyword arguments dictionary:", kwargs)  
display_flexible_inputs(1, 2, status="Active")
```

Output:

```
Positional arguments tuple: (1, 2)  
Keyword arguments dictionary: {'status': 'Active'}
```

PRACTICE PROGRAMS - Functions - *args and **kwargs

Program 1: Write a program to sum any number of arguments using *args.

Input:

```
1, 2, 3, 4
```

Program / Code:

```
def total(*args):  
    return sum(args)  
print(total(1, 2, 3, 4))
```

Output:

```
10
```

Program 2: Write a program to print keyword arguments using `**kwargs`.

Input:

```
name="Ravi", age=22
```

Program / Code:

```
def show(**kwargs):  
    print(kwargs)  
show(name="Ravi", age=22)
```

Output:

```
{'name': 'Ravi', 'age': 22}
```

Program 3: Write a program to find the maximum number using `*args`.

Input:

```
4, 9, 2, 7
```

Program / Code:

```
def find_max(*args):  
    return max(args)  
print(find_max(4, 9, 2, 7))
```

Output:

```
9
```

Program 4: Write a program to display student details using `**kwargs`.

Input:

```
name="Neha", grade="A"
```

Program / Code:

```
def student(**kwargs):  
    for k, v in kwargs.items():  
        print(k, ":", v)  
student(name="Neha", grade="A")
```

Output:

```
name : Neha  
grade : A
```

Program 5: Write a program to combine *args and **kwargs in one function.

Input:

```
1, 2, city="Pune"
```

Program / Code:

```
def info(*args, **kwargs):  
    print(args, kwargs)  
info(1, 2, city="Pune")
```

Output:

```
(1, 2) {'city': 'Pune'}
```

10.3. Functions - Lambda Expressions

Lambda expressions are small, anonymous, single-line functions that can take multiple arguments but evaluate only a single expression.

Syntax:

```
lambda arguments: expression
```

Example Program:

```
# Writing quick, compact formulas on a single line  
multiply_nums = lambda a, b: a * b  
print("Lambda expression multiplication:", multiply_nums(4, 5))
```

Output:

```
Lambda expression multiplication: 20
```

PRACTICE PROGRAMS - Functions - Lambda Expressions

Program 1: Write a program to square a number using lambda.

Input:

```
x = 6
```

Program / Code:

```
square = lambda x: x * x  
print(square(6))
```

Output:

```
36
```

Program 2: Write a program to add two numbers using lambda.

Input:

```
a=7, b=3
```

Program / Code:

```
add = lambda a, b: a + b  
print(add(7, 3))
```

Output:

```
10
```

Program 3: Write a program to find the largest of two numbers using lambda.

Input:

```
a=15, b=9
```

Program / Code:

```
larger = lambda a, b: a if a > b else b  
print(larger(15, 9))
```

Output:

```
15
```

Program 4: Write a program to check if a number is even using lambda.

Input:

```
n=10
```

Program / Code:

```
is_even = lambda n: n % 2 == 0  
print(is_even(10))
```

Output:

```
True
```

Program 5: Write a program to sort a list of tuples using lambda as the key.

Input:

```
pairs = [(1, "b"), (2, "a")]
```

Program / Code:

```
pairs = [(1, "b"), (2, "a")]
pairs.sort(key=lambda x: x[1])
print(pairs)
```

Output:

```
[(2, 'a'), (1, 'b')]
```

10.4. Functions - List Comprehensions

List comprehensions provide a concise syntax to create new lists from existing collections or iterables based on an inline loop.

Syntax:

```
[expression for item in iterable]
```

Example Program:

```
# Building new lists inline with a single line loop
squared_evens = [x*x for x in range(1, 4)]
print("List comprehension result:", squared_evens)
```

Output:

```
List comprehension result: [1, 4, 9]
```

PRACTICE PROGRAMS - Functions - List Comprehensions

Program 1: Write a program to create a list of squares using list comprehension.

Input:

```
None
```

Program / Code:

```
squares = [x*x for x in range(1, 6)]
print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

Program 2: Write a program to filter even numbers using list comprehension.

Input:

```
nums = [1, 2, 3, 4, 5, 6]
```

Program / Code:

```
nums = [1, 2, 3, 4, 5, 6]
evens = [n for n in nums if n % 2 == 0]
print(evens)
```

Output:

```
[2, 4, 6]
```

Program 3: Write a program to convert a list of strings to uppercase using list comprehension.

Input:

```
words = ["hi", "bye"]
```

Program / Code:

```
words = ["hi", "bye"]
upper_words = [w.upper() for w in words]
print(upper_words)
```

Output:

```
['HI', 'BYE']
```

Program 4: Write a program to create a list of cubes of odd numbers using list comprehension.

Input:

```
None
```

Program / Code:

```
cubes = [x**3 for x in range(1, 8) if x % 2 != 0]
print(cubes)
```

Output:

```
[1, 27, 125, 343]
```

Program 5: Write a program to flatten a nested list using list comprehension.

Input:

```
nested = [[1, 2], [3, 4], [5]]
```

Program / Code:

```
nested = [[1, 2], [3, 4], [5]]  
flat = [n for row in nested for n in row]  
print(flat)
```

Output:

```
[1, 2, 3, 4, 5]
```

11.1. OOP - Classes and Objects

A class is a structural blueprint or template for creating objects, which are individual, functional instances of that class.

Syntax:

```
class ClassName:  
    pass  
object_name = ClassName()
```

Example Program:

```
# Defining a basic structural layout and building structural  
object instances  
class CarBlueprint:  
    pass  
my_car_object = CarBlueprint()  
print("Object created:", my_car_object)
```

Output:

```
Object created: <__main__.CarBlueprint object at 0x...>
```

PRACTICE PROGRAMS - OOP - Classes and Objects

Program 1: Write a program to create a class and an object to display a message.

Input:

```
None
```

Program / Code:

```
class Greeting:  
    def hello(self):  
        print("Hello from a class!")  
g = Greeting()  
g.hello()
```

Output:

```
Hello from a class!
```

Program 2: Write a program to create a class Student with a method to display the name.

Input:

```
name = "Kiran"
```

Program / Code:

```
class Student:
    def show(self, name):
        print("Student name:", name)
s = Student()
s.show("Kiran")
```

Output:

```
Student name: Kiran
```

Program 3: Write a program to create multiple objects of a class.

Input:

```
None
```

Program / Code:

```
class Animal:
    def sound(self):
        print("An animal makes a sound")
a1 = Animal()
a2 = Animal()
a1.sound()
a2.sound()
```

Output:

```
An animal makes a sound
An animal makes a sound
```

Program 4: Write a program to create a class with an attribute and print it.

Input:

```
None
```

Program / Code:

```
class Book:
    title = "Python Basics"
b = Book()
print(b.title)
```

Output:

```
Python Basics
```

Program 5: Write a program to define a class Circle with a method to print its name.

Input:

None

Program / Code:

```
class Circle:
    def show_name(self):
        print("This is a Circle shape")
c = Circle()
c.show_name()
```

Output:

This is a Circle shape

11.2. OOP - __init__ & self

The __init__ method is Python's constructor that initializes an object's attributes, while 'self' represents the specific instance being created.

Syntax:

```
def __init__(self, parameters):
    self.attribute = parameters
```

Example Program:

```
# Initializing object properties upon creation using standard
setup methods
class DeviceProfile:
    def __init__(self, core_name):
        self.device_name = core_name
active_phone = DeviceProfile("SmartPhone")
print(active_phone.device_name)
```

Output:

SmartPhone

PRACTICE PROGRAMS - OOP - __init__ & self

Program 1: Write a program to initialize object attributes using __init__.

Input:

```
name = "Pen"
```

Program / Code:

```
class Item:
    def __init__(self, name):
        self.name = name
i = Item("Pen")
print(i.name)
```

Output:

```
Pen
```

Program 2: Write a program to create a class Person with name and age attributes.

Input:

```
name="Meera", age=28
```

Program / Code:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
p = Person("Meera", 28)
print(p.name, p.age)
```

Output:

```
Meera 28
```

Program 3: Write a program to calculate the area of a rectangle using __init__.

Input:

```
l=6, b=4
```

Program / Code:

```
class Rectangle:
    def __init__(self, l, b):
        self.l = l
        self.b = b
    def area(self):
        return self.l * self.b
r = Rectangle(6, 4)
print(r.area())
```

Output:

```
24
```

Program 4: Write a program to create a class Book with title and price attributes.

Input:

```
title="Python 101", price=350
```

Program / Code:

```
class Book:
    def __init__(self, title, price):
        self.title = title
        self.price = price
b = Book("Python 101", 350)
print(b.title, b.price)
```

Output:

```
Python 101 350
```

Program 5: Write a program to update an attribute value using a method.

Input:

```
balance=500, deposit=200
```

Program / Code:

```
class Account:
    def __init__(self, balance):
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
a = Account(500)
a.deposit(200)
print(a.balance)
```

Output:

```
700
```

11.3. OOP - Inheritance

Inheritance allows a child class to inherit attributes and methods from a parent class, promoting code reuse and extensions.

Syntax:

```
class ChildClass(ParentClass):
```

Example Program:

```
# Deriving functional traits from a parent class
class AnimalParent:
    def breathe_action(self): print("Breathing active")
class DogChild(AnimalParent):
    pass
my_dog = DogChild()
my_dog.breathe_action()
```

Output:

```
Breathing active
```

PRACTICE PROGRAMS - OOP - Inheritance

Program 1: Write a program to demonstrate single inheritance.

Input:

None

Program / Code:

```
class Vehicle:
    def move(self):
        print("Vehicle moves")
class Car(Vehicle):
    pass
c = Car()
c.move()
```

Output:

Vehicle moves

Program 2: Write a program to override a parent class method in a child class.

Input:

None

Program / Code:

```
class Parent:
    def show(self):
        print("Parent method")
class Child(Parent):
    def show(self):
        print("Child method")
c = Child()
c.show()
```

Output:

Child method

Program 3: Write a program to demonstrate multilevel inheritance.

Input:

None

Program / Code:

```
class A:
    def show(self): print("Class A")
class B(A):
    pass
class C(B):
    pass
C().show()
```

Output:

Class A

Program 4: Write a program to demonstrate multiple inheritance.

Input:

None

Program / Code:

```
class Father:
    def skill1(self): print("Driving")
class Mother:
    def skill2(self): print("Cooking")
class Child(Father, Mother):
    pass
k = Child()
k.skill1()
k.skill2()
```

Output:

Driving
Cooking

Program 5: Write a program to call a parent class method using super().

Input:

None

Program / Code:

```
class Parent:
    def greet(self):
        print("Hello from Parent")
class Child(Parent):
    def greet(self):
        super().greet()
        print("Hello from Child")
Child().greet()
```

Output:

```
Hello from Parent
Hello from Child
```

11.4. OOP - Polymorphism

Polymorphism allows different classes to share the same method name but implement their own specific behaviors.

Syntax:

```
# Multiple classes implementing the same method name
```

Example Program:

```
# Executing unique methods through a shared standard interface
naming style
class Truck:
    def drive(self): return "Driving a heavy vehicle"
class Bike:
    def drive(self): return "Riding a light bicycle"
print(Truck().drive())
```

Output:

```
Driving a heavy vehicle
```

PRACTICE PROGRAMS - OOP - Polymorphism

Program 1: Write a program to demonstrate polymorphism using two classes with the same method name.

Input:

None

Program / Code:

```
class Cat:
    def sound(self): return "Meow"
class Dog:
    def sound(self): return "Woof"
for animal in (Cat(), Dog()):
    print(animal.sound())
```

Output:

Meow
Woof

Program 2: Write a program to demonstrate method overriding as polymorphism.

Input:

None

Program / Code:

```
class Shape:
    def area(self): return 0
class Square(Shape):
    def area(self): return 16
print(Square().area())
```

Output:

16

Program 3: Write a program to use polymorphism with a common function for different objects.

Input:

None

Program / Code:

```
class Cat:
    def sound(self): return "Meow"
class Dog:
    def sound(self): return "Woof"
def make_sound(animal):
    print(animal.sound())
make_sound(Cat())
```

Output:

Meow

Program 4: Write a program to demonstrate polymorphism using len() on different data types.

Input:

None

Program / Code:

```
print(len("python"))
print(len([1, 2, 3, 4]))
```

Output:

6
4

Program 5: Write a program to demonstrate polymorphism with area calculation of different shapes.

Input:

None

Program / Code:

```
class Circle:
    def area(self): return 3.14 * 5 * 5
class Square:
    def area(self): return 5 * 5
for shape in (Circle(), Square()):
    print(shape.area())
```

Output:

78.5
25

11.5. OOP - Encapsulation

Encapsulation restricts direct access to an object's components by prefixing attribute names with underscores to prevent external modifications.

Syntax:

```
self.__private_variable = value
```

Example Program:

```
# Protecting internal class properties from external access
class SecureVault:
    def __init__(self):
        self.__secret_key = "99XF"
    def get_key(self):
        return self.__secret_key
my_vault = SecureVault()
print(my_vault.get_key())
```

Output:

99XF

PRACTICE PROGRAMS - OOP - Encapsulation

Program 1: Write a program to demonstrate encapsulation using a private variable.

Input:

None

Program / Code:

```
class Account:
    def __init__(self):
        self.__pin = 1234
account = Account()
print(account._Account__pin)
```

Output:

1234

Program 2: Write a program to access a private variable using a getter method.

Input:

salary=45000

Program / Code:

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary
    def get_salary(self):
        return self.__salary
e = Employee(45000)
print(e.get_salary())
```

Output:

45000

Program 3: Write a program to update a private variable using a setter method.

Input:

salary=30000, new_salary=35000

Program / Code:

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary
    def set_salary(self, new_salary):
        self.__salary = new_salary
    def get_salary(self):
        return self.__salary
e = Employee(30000)
e.set_salary(35000)
print(e.get_salary())
```

Output:

35000

Program 4: Write a program to demonstrate protected members using a single underscore.

Input:

None

Program / Code:

```
class Base:
    def __init__(self):
        self._status = "active"
class Derived(Base):
    def show(self):
        print(self._status)
Derived().show()
```

Output:

active

Program 5: Write a program to hide bank balance details using encapsulation.

Input:

```
balance=15000
```

Program / Code:

```
class Bank:
    def __init__(self, balance):
        self.__balance = balance
    def show_balance(self):
        print("Balance:", self.__balance)
b = Bank(15000)
b.show_balance()
```

Output:

```
Balance: 15000
```

11.6. OOP - Abstraction

Abstraction hides complex internal implementation details and only exposes essential features using abstract base classes and methods.

Syntax:

```
from abc import ABC, abstractmethod
```

Example Program:

```
# Enforcing structural design requirements using abstract
frameworks
from abc import ABC, abstractmethod
class AbstractForm(ABC):
    @abstractmethod
    def click_action(self): pass
```

Output:

```
(No direct output - defines an abstract template for subclasses)
```

PRACTICE PROGRAMS - OOP - Abstraction

Program 1: Write a program to create an abstract class with an abstract method.

Input:

None

Program / Code:

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): pass
print("Abstract class defined")
```

Output:

Abstract class defined

Program 2: Write a program to implement an abstract method in a subclass.

Input:

None

Program / Code:

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): pass
class Square(Shape):
    def area(self):
        return 4 * 4
print(Square().area())
```

Output:

16

Program 3: Write a program to demonstrate abstraction using a Shape abstract class.

Input:

None

Program / Code:

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def draw(self): pass
class Triangle(Shape):
    def draw(self):
        print("Drawing a triangle")
Triangle().draw()
```

Output:

Drawing a triangle

Program 4: Write a program to demonstrate abstraction with multiple subclasses.

Input:

None

Program / Code:

```
from abc import ABC, abstractmethod
class Payment(ABC):
    @abstractmethod
    def pay(self): pass
class CardPayment(Payment):
    def pay(self):
        print("Paid using card")
class CashPayment(Payment):
    def pay(self):
        print("Paid using cash")
CardPayment().pay()
CashPayment().pay()
```

Output:

Paid using card
Paid using cash

Program 5: Write a program to show that an abstract class cannot be instantiated.

Input:

None

Program / Code:

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): pass
try:
    s = Shape()
except TypeError as e:
    print("Error:", e)
```

Output:

```
Error: Can't instantiate abstract class Shape with abstract
method area
```

12.1. Advanced Topics - File Operations

File operations allow you to open, read, write, and close physical external files stored on the computer disk safely.

Syntax:

```
with open("filename.txt", "mode") as file:
```

Example Program:

```
# Opening and writing text lines into a document safely with
context managers
with open("test_file.txt", "w") as target_file:
    target_file.write("Python writing sample.")
```

Output:

```
(File "test_file.txt" is created/overwritten with the given text)
```

PRACTICE PROGRAMS - Advanced Topics - File Operations

Program 1: Write a program to write text into a file.

Input:

```
text = "Learning Python file handling"
```

Program / Code:

```
with open("notes.txt", "w") as f:
    f.write("Learning Python file handling")
```

Output:

```
(notes.txt is created with the text written into it)
```

Program 2: Write a program to read content from a file.

Input:

```
notes.txt contains: Learning Python file handling
```

Program / Code:

```
with open("notes.txt", "r") as f:
    print(f.read())
```

Output:

```
Learning Python file handling
```

Program 3: Write a program to append text to an existing file.

Input:

None

Program / Code:

```
with open("notes.txt", "a") as f:  
    f.write("\nAppended line")
```

Output:

(Appended line is added at the end of notes.txt)

Program 4: Write a program to read a file line by line.

Input:

notes.txt has multiple lines

Program / Code:

```
with open("notes.txt", "r") as f:  
    for line in f:  
        print(line.strip())
```

Output:

Learning Python file handling
Appended line

Program 5: Write a program to check if a file exists before reading it.

Input:

None

Program / Code:

```
import os  
if os.path.exists("notes.txt"):  
    print("File exists")  
else:  
    print("File not found")
```

Output:

File exists

12.2. Advanced Topics - Exception Handling

Exception handling isolates code blocks using try, except, and finally to catch runtime errors gracefully without crashing the app.

Syntax:

```
try:
    # Code
except Exception as e:
    # Handle error
```

Example Program:

```
# Handling division faults elegantly using try-except guards
try:
    division_calc = 10 / 0
except ZeroDivisionError:
    print("Cannot compute division by zero!")
```

Output:

```
Cannot compute division by zero!
```

PRACTICE PROGRAMS - Advanced Topics - Exception Handling

Program 1: Write a program to handle a division by zero exception.

Input:

```
None
```

Program / Code:

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("Division by zero not allowed")
```

Output:

```
Division by zero not allowed
```

Program 2: Write a program to handle an invalid type conversion using exception handling.

Input:

```
"abc"
```

Program / Code:

```
try:
    num = int("abc")
except ValueError:
    print("Invalid number format")
```

Output:

```
Invalid number format
```

Program 3: Write a program to use try, except and finally together.

Input:

None

Program / Code:

```
try:
    print(5 / 1)
except ZeroDivisionError:
    print("Error occurred")
finally:
    print("Execution complete")
```

Output:

```
5.0
Execution complete
```

Program 4: Write a program to handle multiple exceptions.

Input:

lst = [1, 2]

Program / Code:

```
try:
    lst = [1, 2]
    print(lst[5])
except IndexError:
    print("Index out of range")
except ValueError:
    print("Value error")
```

Output:

```
Index out of range
```

Program 5: Write a program to raise a custom exception.

Input:

```
age=15
```

Program / Code:

```
def check_age(age):  
    if age < 18:  
        raise ValueError("Age must be 18 or above")  
    return "Valid age"  
try:  
    print(check_age(15))  
except ValueError as e:  
    print(e)
```

Output:

```
Age must be 18 or above
```

12.3. Advanced Topics - Generators

Generators are memory-efficient functions that return an iterator loop stream sequentially using the 'yield' keyword instead of 'return'.

Syntax:

```
def generator_name():  
    yield value
```

Example Program:

```
# Yielding single output values on-demand to save computer memory  
def stream_numbers():  
    yield 1  
    yield 2  
number_iterator = stream_numbers()  
print(next(number_iterator))
```

Output:

```
1
```

PRACTICE PROGRAMS - Advanced Topics - Generators

Program 1: Write a program to create a generator function to yield numbers.

Input:

None

Program / Code:

```
def gen_nums():  
    yield 10  
    yield 20  
    yield 30  
for n in gen_nums():  
    print(n)
```

Output:

```
10  
20  
30
```

Program 2: Write a program to create a generator for even numbers.

Input:

limit=10

Program / Code:

```
def even_gen(limit):  
    for i in range(2, limit + 1, 2):  
        yield i  
for n in even_gen(10):  
    print(n)
```

Output:

```
2  
4  
6  
8  
10
```

Program 3: Write a program to use a generator to generate squares of numbers.

Input:

n=5

Program / Code:

```
def squares(n):  
    for i in range(1, n + 1):  
        yield i * i  
print(list(squares(5)))
```

Output:

[1, 4, 9, 16, 25]

Program 4: Write a program to iterate a generator using a for loop.

Input:

None

Program / Code:

```
def colors():  
    yield "red"  
    yield "blue"  
for c in colors():  
    print(c)
```

Output:

red
blue

Program 5: Write a program to create a generator with a break condition when iterating.

Input:

None

Program / Code:

```
def counter():
    n = 1
    while True:
        yield n
        n += 1
for val in counter():
    if val > 5:
        break
    print(val)
```

Output:

```
1
2
3
4
5
```

12.4. Advanced Topics - Decorators

Decorators are wrappers that dynamically modify or extend the behavior of a target function without modifying its permanent source code.

Syntax:

```
@decorator_name
def target_function():
```

Example Program:

```
# Injecting secondary operations cleanly using functional
wrappers
def log_decorator(original_func):
    def wrapper_logic():
        print("Log message text before call")
        original_func()
    return wrapper_logic
@log_decorator
def say_hi():
    print("Hi!")
say_hi()
```

Output:

```
Log message text before call  
Hi!
```

PRACTICE PROGRAMS - Advanced Topics - Decorators

Program 1: Write a program to create a simple decorator to print before and after a function call.

Input:

None

Program / Code:

```
def my_decorator(func):  
    def wrapper():  
        print("Before")  
        func()  
        print("After")  
    return wrapper  
@my_decorator  
def greet():  
    print("Hello")  
greet()
```

Output:

```
Before  
Hello  
After
```

Program 2: Write a program to create a decorator to calculate execution time.

Input:

None

Program / Code:

```
import time
def timer(func):
    def wrapper():
        start = time.time()
        func()
        print("Time taken:", round(time.time() - start, 4))
    return wrapper
@timer
def task():
    print("Task running")
task()
```

Output:

```
Task running
Time taken: 0.0 (approx.)
```

Program 3: Write a program to create a decorator to convert output to uppercase.

Input:

None

Program / Code:

```
def upper_case(func):
    def wrapper():
        return func().upper()
    return wrapper
@upper_case
def message():
    return "hello python"
print(message())
```

Output:

```
HELLO PYTHON
```

Program 4: Write a program to create a decorator that checks user login before running a function.

Input:

```
is_logged_in=False
```

Program / Code:

```
def login_required(func):
    def wrapper(is_logged_in):
        if is_logged_in:
            func()
        else:
            print("Please log in first")
    return wrapper
@login_required
def dashboard():
    print("Welcome to dashboard")
dashboard(False)
```

Output:

```
Please log in first
```

Program 5: Write a program to apply two decorators to a single function.

Input:

```
None
```

Program / Code:

```
def bold(func):
    def wrapper():
        return "***" + func() + "***"
    return wrapper
def italic(func):
    def wrapper():
        return "_" + func() + "_"
    return wrapper
@bold
@italic
def text():
    return "Python"
print(text())
```

Output:

```
**_Python_**
```

12.5. Advanced Topics - Regular Expressions (re)

Regular Expressions (re) provide search parameters and matching tools to scan, filter, and modify text strings based on specific character patterns.

Syntax:

```
import re
match = re.search(pattern, string)
```

Example Program:

```
# Extracting text characters using string search algorithms
import re
text_block = "User phone code is 4567"
found_digits = re.findall(r"\d+", text_block)
print("Extracted pattern digits list:", found_digits)
```

Output:

```
Extracted pattern digits list: ['4567']
```

PRACTICE PROGRAMS - Advanced Topics - Regular Expressions (re)

Program 1: Write a program to extract all digits from a string using re.

Input:

```
text = "Order 45 has 3 items"
```

Program / Code:

```
import re
text = "Order 45 has 3 items"
print(re.findall(r"\d+", text))
```

Output:

```
['45', '3']
```

Program 2: Write a program to check if a string matches an email pattern.

Input:

```
email = "user@example.com"
```

Program / Code:

```
import re
email = "user@example.com"
pattern = r"^[a-zA-Z0-9_]+@[a-zA-Z0-9]+\.[a-z]+$"
print(bool(re.match(pattern, email)))
```

Output:

```
True
```

Program 3: Write a program to replace all spaces in a string with underscores using re.

Input:

```
text = "Python is fun"
```

Program / Code:

```
import re
text = "Python is fun"
print(re.sub(r" ", "_", text))
```

Output:

```
Python_is_fun
```

Program 4: Write a program to find all words in a string using re.

Input:

```
text = "Learn Python fast"
```

Program / Code:

```
import re
text = "Learn Python fast"
print(re.findall(r"[A-Za-z]+", text))
```

Output:

```
['Learn', 'Python', 'fast']
```

Program 5: Write a program to validate a 10-digit phone number using re.

Input:

```
phone = "9876543210"
```

Program / Code:

```
import re
phone = "9876543210"
pattern = r"^\d{10}$"
print(bool(re.match(pattern, phone)))
```

Output:

```
True
```

12.6. Advanced Topics - Virtual Environments (venv) & pip

Virtual environments isolate dependencies for individual projects, while pip downloads and manages external third-party libraries.

Syntax:

```
python -m venv environment_name
pip install package_name
```

Example Program:

```
# Executing terminal setup commands to build secure, isolated
project scopes
# python -m venv development_env
# pip install requests
```

Output:

```
(Creates an isolated "development_env" folder and installs the
requests library into it)
```

PRACTICE PROGRAMS - Advanced Topics - Virtual Environments (venv) & pip

Program 1: Write a program to check the installed pip version from within Python.

Input:

None

Program / Code:

```
import subprocess, sys
result = subprocess.run([sys.executable, "-m", "pip", "--version"],
capture_output=True, text=True)
print(result.stdout.strip())
```

Output:

```
pip 24.x.x from ... (python 3.12)
```

Program 2: Write a program to list installed packages using pip via subprocess.

Input:

None

Program / Code:

```
import subprocess, sys
result = subprocess.run([sys.executable, "-m", "pip", "list"],
capture_output=True, text=True)
print(result.stdout)
```

Output:

```
Package Version
-----
pip 24.x
...
```

Program 3: Write a program to check the Python version being used.

Input:

None

Program / Code:

```
import platform
print(platform.python_version())
```

Output:

3.12.x

Program 4: Write a program to check if a package is installed using importlib.

Input:

None

Program / Code:

```
import importlib.util
found = importlib.util.find_spec("requests")
print("Installed" if found else "Not installed")
```

Output:

Installed

Program 5: Write a program to print the current working directory before setting up a virtual environment.

Input:

None

Program / Code:

```
import os
print(os.getcwd())
```

Output:

/home/user/project (path depends on your system)

13. Interview Programs

A set of 40 frequently-asked, mixed-topic interview-style programs combining loops, strings, lists, dictionaries and recursion.

INTERVIEW PROGRAMS (40 Questions)

Program 1: Write a program to check whether a given number is a palindrome.

Input:

```
num = 12321
```

Program / Code:

```
num = 12321
temp = num
rev = 0
while temp > 0:
    rev = rev * 10 + temp % 10
    temp //= 10
print(num == rev)
```

Output:

```
True
```

Program 2: Write a program to find the factorial of a number using recursion.

Input:

```
n = 5
```

Program / Code:

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
print(factorial(5))
```

Output:

```
120
```

Program 3: Write a program to check whether a given string is a palindrome.

Input:

```
text = "madam"
```

Program / Code:

```
text = "madam"  
print(text == text[::-1])
```

Output:

```
True
```

Program 4: Write a program to find the Fibonacci series up to n terms.

Input:

```
n = 6
```

Program / Code:

```
n = 6  
a, b = 0, 1  
for _ in range(n):  
    print(a, end=" ")  
    a, b = b, a + b
```

Output:

```
0 1 1 2 3 5
```

Program 5: Write a program to find the largest element in a list without using max().

Input:

```
nums = [23, 67, 12, 89, 45]
```

Program / Code:

```
nums = [23, 67, 12, 89, 45]  
largest = nums[0]  
for n in nums:  
    if n > largest:  
        largest = n  
print(largest)
```

Output:

```
89
```

Program 6: Write a program to count the frequency of each element in a list.

Input:

```
nums = [1, 2, 2, 3, 1, 3, 3]
```

Program / Code:

```
nums = [1, 2, 2, 3, 1, 3, 3]
freq = {}
for n in nums:
    freq[n] = freq.get(n, 0) + 1
print(freq)
```

Output:

```
{1: 2, 2: 2, 3: 3}
```

Program 7: Write a program to check whether a given number is an Armstrong number.

Input:

```
num = 153
```

Program / Code:

```
num = 153
total = sum(int(d) ** len(str(num)) for d in str(num))
print(total == num)
```

Output:

```
True
```

Program 8: Write a program to remove duplicate elements from a list without using set().

Input:

```
nums = [1, 2, 2, 3, 4, 4, 5]
```

Program / Code:

```
nums = [1, 2, 2, 3, 4, 4, 5]
result = []
for n in nums:
    if n not in result:
        result.append(n)
print(result)
```

Output:

```
[1, 2, 3, 4, 5]
```

Program 9: Write a program to find the second largest number in a list.

Input:

```
nums = [34, 67, 12, 89, 45]
```

Program / Code:

```
nums = [34, 67, 12, 89, 45]
unique_sorted = sorted(set(nums), reverse=True)
print(unique_sorted[1])
```

Output:

```
67
```

Program 10: Write a program to check whether two strings are anagrams of each other.

Input:

```
s1 = "listen", s2 = "silent"
```

Program / Code:

```
s1 = "listen"
s2 = "silent"
print(sorted(s1) == sorted(s2))
```

Output:

```
True
```

INTERVIEW PROGRAMS – QUESTIONS 11–40

Program 11: Write a program to reverse a string without using slicing.

Input:

```
text = "python"
```

Program / Code:

```
text = "python"
rev = ""
for ch in text:
    rev = ch + rev
print(rev)
```

Output:

```
nohtyp
```

Program 12: Write a program to count vowels and consonants in a string.

Input:

```
text = "Python Programming"
```

Program / Code:

```
text = "Python Programming"
vowels = consonants = 0
for ch in text.lower():
    if ch.isalpha():
        if ch in "aeiou":
            vowels += 1
        else:
            consonants += 1
print("Vowels:", vowels)
print("Consonants:", consonants)
```

Output:

```
Vowels: 4
Consonants: 13
```

Program 13: Write a program to find the first non-repeating character in a string.

Input:

```
text = "swiss"
```

Program / Code:

```
text = "swiss"
for ch in text:
    if text.count(ch) == 1:
        print(ch)
        break
```

Output:

```
w
```

Program 14: Write a program to count the frequency of each character in a string.

Input:

```
text = "banana"
```

Program / Code:

```
text = "banana"
freq = {}
for ch in text:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

Output:

```
{'b': 1, 'a': 3, 'n': 2}
```

Program 15: Write a program to remove duplicate characters from a string while preserving order.

Input:

```
text = "programming"
```

Program / Code:

```
text = "programming"
result = ""
for ch in text:
    if ch not in result:
        result += ch
print(result)
```

Output:

```
progamin
```

Program 16: Write a program to find two numbers in a list whose sum equals a target.

Input:

```
nums = [2, 7, 11, 15], target = 9
```

Program / Code:

```
nums = [2, 7, 11, 15]
target = 9
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] + nums[j] == target:
            print(nums[i], nums[j])
```

Output:

```
2 7
```

Program 17: Write a program to find the missing number from 1 to n.

Input:

```
nums = [1, 2, 3, 5, 6]
```

Program / Code:

```
nums = [1, 2, 3, 5, 6]
n = len(nums) + 1
expected = n * (n + 1) // 2
print(expected - sum(nums))
```

Output:

```
4
```

Program 18: Write a program to find common elements in two lists without duplicates.

Input:

```
a = [1, 2, 2, 3, 4], b = [2, 3, 3, 5]
```

Program / Code:

```
a = [1, 2, 2, 3, 4]
b = [2, 3, 3, 5]
common = []
for x in a:
    if x in b and x not in common:
        common.append(x)
print(common)
```

Output:

```
[2, 3]
```

Program 19: Write a program to merge two sorted lists into one sorted list.

Input:

```
a = [1, 3, 5], b = [2, 4, 6]
```

Program / Code:

```
a = [1, 3, 5]
b = [2, 4, 6]
i = j = 0
merged = []
while i < len(a) and j < len(b):
    if a[i] <= b[j]:
        merged.append(a[i]); i += 1
    else:
        merged.append(b[j]); j += 1
merged.extend(a[i:]); merged.extend(b[j:])
print(merged)
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

Program 20: Write a program to sort a list without using sort() or sorted().

Input:

```
nums = [5, 2, 9, 1, 3]
```

Program / Code:

```
nums = [5, 2, 9, 1, 3]
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] > nums[j]:
            nums[i], nums[j] = nums[j], nums[i]
print(nums)
```

Output:

```
[1, 2, 3, 5, 9]
```

Program 21: Write a program to perform a linear search for an element in a list.

Input:

```
nums = [10, 20, 30, 40], key = 30
```

Program / Code:

```
nums = [10, 20, 30, 40]
key = 30
for i, value in enumerate(nums):
    if value == key:
        print("Found at index:", i)
        break
else:
    print("Not found")
```

Output:

```
Found at index: 2
```

Program 22: Write a program to perform binary search on a sorted list.

Input:

```
nums = [10, 20, 30, 40, 50], key = 40
```

Program / Code:

```
nums = [10, 20, 30, 40, 50]
key = 40
low, high = 0, len(nums) - 1
while low <= high:
    mid = (low + high) // 2
    if nums[mid] == key:
        print("Found at index:", mid)
        break
    elif nums[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
```

Output:

```
Found at index: 3
```

Program 23: Write a program to print all prime numbers in a given range.

Input:

```
start = 10, end = 30
```

Program / Code:

```
start, end = 10, 30
for n in range(start, end + 1):
    if n < 2:
        continue
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            break
    else:
        print(n, end=" ")
```

Output:

```
11 13 17 19 23 29
```

Program 24: Write a program to find the greatest common divisor (GCD) of two numbers.

Input:

```
a = 48, b = 18
```

Program / Code:

```
a, b = 48, 18
while b:
    a, b = b, a % b
print(a)
```

Output:

```
6
```

Program 25: Write a program to find the least common multiple (LCM) of two numbers.

Input:

```
a = 12, b = 18
```

Program / Code:

```
a, b = 12, 18
x, y = a, b
while y:
    x, y = y, x % y
gcd = x
print((a * b) // gcd)
```

Output:

```
36
```

Program 26: Write a program to check whether a string is a palindrome ignoring spaces and case.

Input:

```
text = "Never Odd Or Even"
```

Program / Code:

```
text = "Never Odd Or Even"
clean = "".join(text.lower().split())
print(clean == clean[::-1])
```

Output:

```
True
```

Program 27: Write a program to check whether brackets are balanced using a stack.

Input:

```
text = "{[()]}"
```

Program / Code:

```
text = "{[()]}"
pairs = {")": "(", "]" : "[", "}": "{"}
stack = []
for ch in text:
    if ch in "([{":
        stack.append(ch)
    elif ch in pairs and (not stack or stack.pop() != pairs[ch]):
        print(False)
        break
else:
    print(not stack)
```

Output:

```
True
```

Program 28: Write a program to validate a string containing only balanced parentheses.

Input:

```
text = "(()())"
```

Program / Code:

```
text = "(()())"
balance = 0
for ch in text:
    balance += 1 if ch == "(" else -1
    if balance < 0:
        break
print(balance == 0)
```

Output:

```
True
```

Program 29: Write a program to count the frequency of each word in a sentence.

Input:

```
text = "python is easy and python is powerful"
```

Program / Code:

```
text = "python is easy and python is powerful"
words = text.lower().split()
freq = {}
for word in words:
    freq[word] = freq.get(word, 0) + 1
print(freq)
```

Output:

```
{'python': 2, 'is': 2, 'easy': 1, 'and': 1, 'powerful': 1}
```

Program 30: Write a program to find the longest word in a sentence.

Input:

```
text = "Python programming is interesting"
```

Program / Code:

```
text = "Python programming is interesting"
words = text.split()
longest = words[0]
for word in words[1:]:
    if len(word) > len(longest):
        longest = word
print(longest)
```

Output:

```
programming
```

Program 31: Write a program to find the second smallest distinct number in a list.

Input:

```
nums = [8, 3, 5, 3, 1, 8]
```

Program / Code:

```
nums = [8, 3, 5, 3, 1, 8]
unique = sorted(set(nums))
print(unique[1])
```

Output:

```
3
```

Program 32: Write a program to rotate a list to the right by k positions.

Input:

```
nums = [1, 2, 3, 4, 5], k = 2
```

Program / Code:

```
nums = [1, 2, 3, 4, 5]
k = 2
k %= len(nums)
rotated = nums[-k:] + nums[:-k]
print(rotated)
```

Output:

```
[4, 5, 1, 2, 3]
```

Program 33: Write a program to move all zeros to the end of a list.

Input:

```
nums = [0, 1, 0, 3, 12]
```

Program / Code:

```
nums = [0, 1, 0, 3, 12]
result = [x for x in nums if x != 0]
result += [0] * (len(nums) - len(result))
print(result)
```

Output:

```
[1, 3, 12, 0, 0]
```

Program 34: Write a program to find the transpose of a matrix.

Input:

```
matrix = [[1, 2, 3], [4, 5, 6]]
```

Program / Code:

```
matrix = [[1, 2, 3], [4, 5, 6]]
transpose = [[row[i] for row in matrix] for i in range(len(matrix[0]))]
print(transpose)
```

Output:

```
[[1, 4], [2, 5], [3, 6]]
```

Program 35: Write a program to find the sum of both diagonals of a square matrix.

Input:

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Program / Code:

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
n = len(matrix)
total = sum(matrix[i][i] + matrix[i][n - 1 - i] for i in range(n))
if n % 2 == 1:
    total -= matrix[n // 2][n // 2]
print(total)
```

Output:

```
25
```

Program 36: Write a program to flatten a nested list without using itertools.

Input:

```
nested = [[1, 2], [3, 4], [5, 6]]
```

Program / Code:

```
nested = [[1, 2], [3, 4], [5, 6]]
flat = []
for row in nested:
    for value in row:
        flat.append(value)
print(flat)
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

Program 37: Write a program to merge two dictionaries into one dictionary.

Input:

```
a = {"x": 1, "y": 2}, b = {"z": 3}
```

Program / Code:

```
a = {"x": 1, "y": 2}
b = {"z": 3}
merged = a.copy()
merged.update(b)
print(merged)
```

Output:

```
{'x': 1, 'y': 2, 'z': 3}
```

Program 38: Write a program to group a list of words into anagram groups.

Input:

```
words = ["eat", "tea", "tan", "ate", "nat", "bat"]
```

Program / Code:

```
words = ["eat", "tea", "tan", "ate", "nat", "bat"]
groups = {}
for word in words:
    key = "".join(sorted(word))
    groups.setdefault(key, []).append(word)
print(list(groups.values()))
```

Output:

```
[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

Program 39: Write a program to find the sum of digits of a number using recursion.

Input:

```
n = 12345
```

Program / Code:

```
def digit_sum(n):  
    if n < 10:  
        return n  
    return n % 10 + digit_sum(n // 10)  
  
print(digit_sum(12345))
```

Output:

```
15
```

Program 40: Write a program to generate Fibonacci numbers using recursion.

Input:

```
n = 7
```

Program / Code:

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    return fibonacci(n - 1) + fibonacci(n - 2)  
  
for i in range(7):  
    print(fibonacci(i), end=" ")
```

Output:

```
0 1 1 2 3 5 8
```